Motivation

Let's return to our well known classic - the *Iris* dataset. Imagine you're tasked with writing a program for classifying the three species. You'd probably perform some basic data exploration and visualization...

In [50]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

#kind of important - we have to talk about this a bit later again ....
my_random_state = 10
```

In [51]:
```
iris_data = sns.load_dataset('iris')
iris_data.sample(n=5)
```

Out[51]:

	sepal_length	sepal_width	petal_length	petal_width	species
56	6.3	3.3	4.7	1.6	versicolor
26	5.0	3.4	1.6	0.4	setosa
149	5.9	3.0	5.1	1.8	virginica
39	5.1	3.4	1.5	0.2	setosa
46	5.1	3.8	1.6	0.2	setosa

In [52]:
```
iris_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal_length  150 non-null    float64
 1   sepal_width   150 non-null    float64
 2   petal_length  150 non-null    float64
 3   petal_width   150 non-null    float64
 4   species       150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

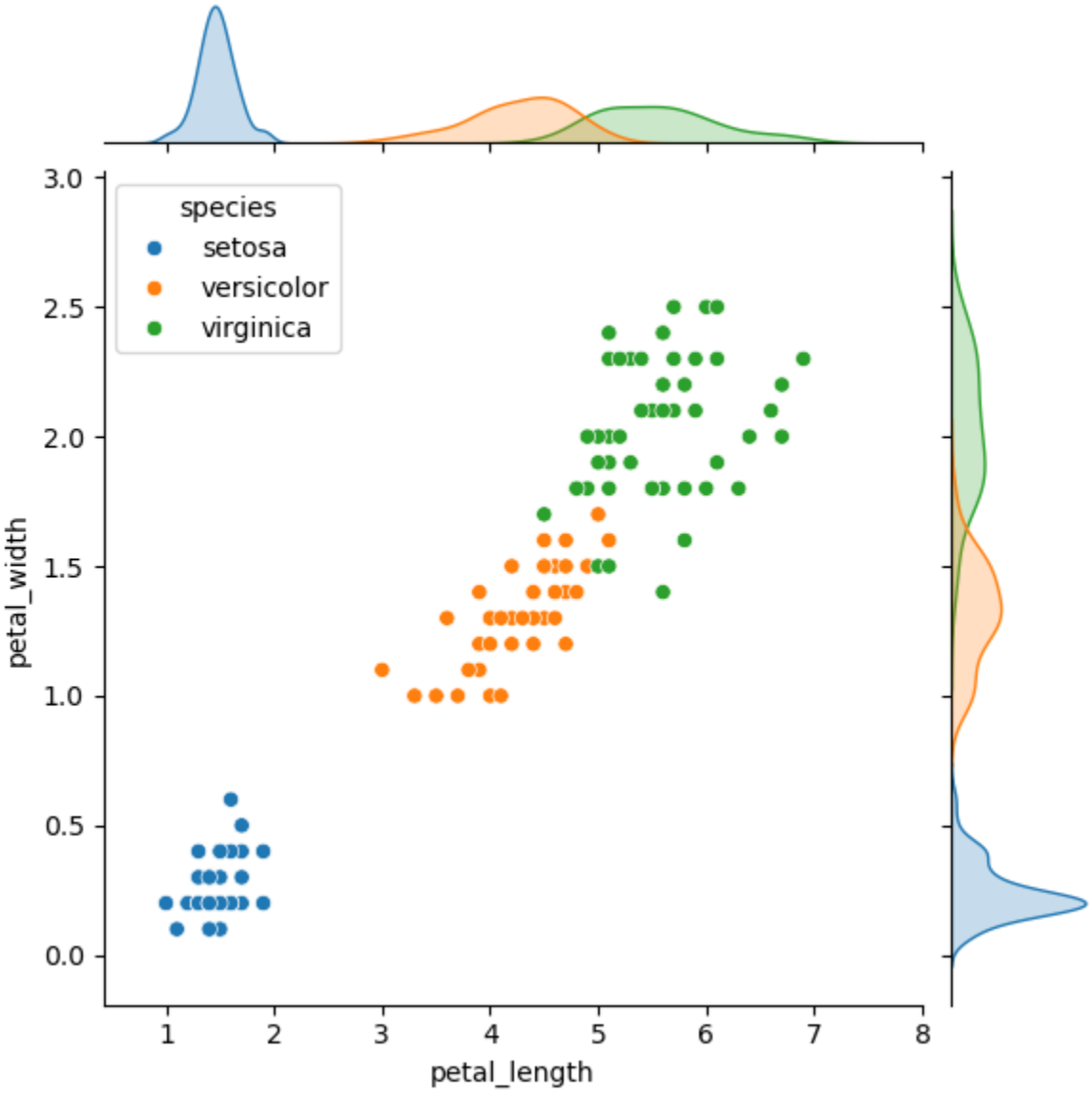
In [53]:
```
iris_data.groupby('species').agg(['min', 'mean', 'max'])
```

Out[53]:

	sepal_length			sepal_width			petal_length			petal_width		
	min	mean	max	min	mean	max	min	mean	max	min	mean	max
species												
setosa	4.3	5.006	5.8	2.3	3.428	4.4	1.0	1.462	1.9	0.1	0.246	0.6
versicolor	4.9	5.936	7.0	2.0	2.770	3.4	3.0	4.260	5.1	1.0	1.326	1.8
virginica	4.9	6.588	7.9	2.2	2.974	3.8	4.5	5.552	6.9	1.4	2.026	2.5

In [54]:
```
sns.jointplot(data=iris_data, x='petal_length', y='petal_width', hue='specie
```

Out[54]:
```
<seaborn.axisgrid.JointGrid at 0x7f7f780db610>
```



...and then come up with a solution (probably) similar to the following *Java* example:

```
if(petalLength <= 2) {
    return Species.SETOSA;
```

```
    }
    else {
        if(5 <= petalLength) {
            return Species.VERSICOLOR;
        }
        else {
            if(1.8 < petalWidth) {
                return Species.VIRGINICA;
            }
            else {
                return Species.VERSICOLOR;
            }
        }
    }
```

The exemplary implementation above resembles a *binary tree*: at each node we ask a *yes/no*-question (e.g. *Is the petal length greater than or equal to 5?*) and either continue asking or ascertain the result.

This analogy - with the questions being the *decisions* and the results the *leaves* - is the base for ***Decision Trees***, a widely used ***supervised*** *machine learning* algorithm, popular for its simplicity and interpretability. Before we dive deeper into the technical details, let's try it out on the *Iris* dataset:

1. We import the relevant classes/modules from *scikit_learn*.
    - We use the `Classifier` since we're dealing with a ***classification*** problem.
2. We segregate the ***features*** into variable `X` and the ***label*** into variable `y`.
3. We ***split*** the data into two parts, one for ***training*** and one for ***testing***.
4. We ***train*** the *decision tree* ***model*** using `fit`, supplying the training data as input.
5. We use the model to ***predict*** the labels using the features from the test data.
6. We calculate the ***accuracy*** (*number of correct predictions* divided by the *total number of predictions*) to ***evaluate*** our model.

In [55]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
```

In [56]:
```
X = iris_data[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y = iris_data['species']

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=21)

dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, y_train)

y_pred = dt_model.predict(X_test)

accuracy_score = metrics.accuracy_score(y_test, y_pred)
print('Accuracy: {:.2%}'.format(accuracy_score))
```

```
Accuracy: 92.11%
```

Above 90% accuracy, without adjusting any ***hyperparameters***!

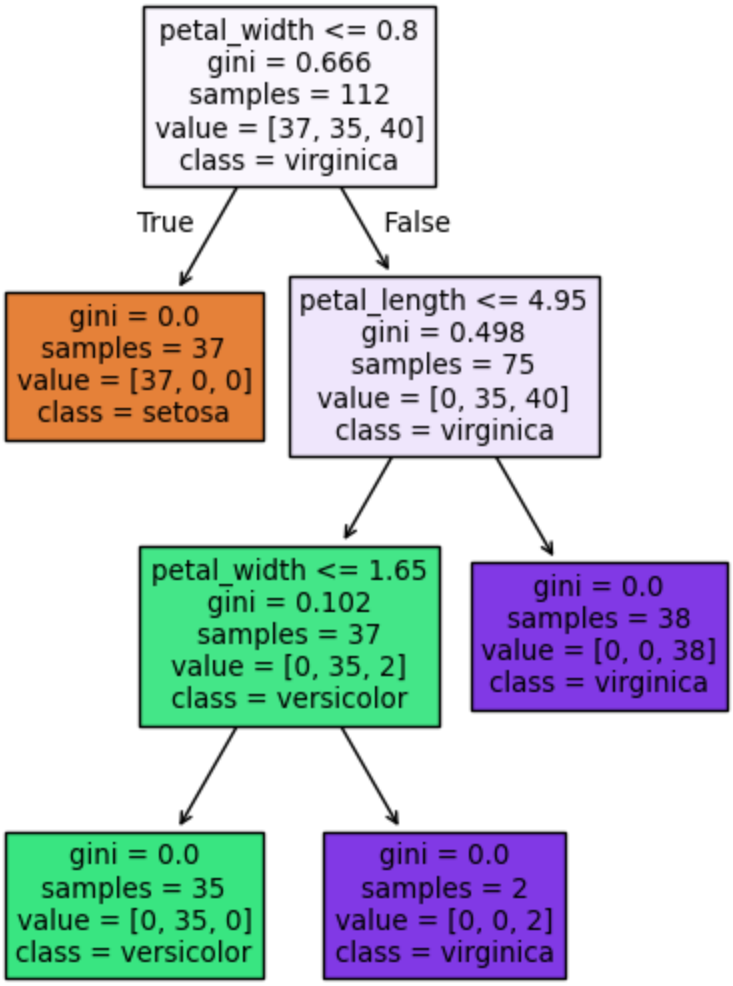
We can take a look at the structure of the *decision tree* as follows - demonstrating its *interpretability*:

In [57]:
```
from sklearn import tree
```

In [58]:
```
text_representation = tree.export_text(dt_model, feature_names=list(iris_dat
print(text_representation)
```

```
|--- petal_width <= 0.80
|   |--- class: setosa
|--- petal_width >  0.80
|   |--- petal_length <= 4.95
|   |   |--- petal_width <= 1.65
|   |   |   |--- class: versicolor
|   |   |--- petal_width >  1.65
|   |   |   |--- class: virginica
|   |--- petal_length >  4.95
|   |   |--- class: virginica
```

In [59]:
```
fig = plt.figure(figsize=(5,7))
tree.plot_tree(dt_model, feature_names=iris_data.columns[:-1], class_names=i
```



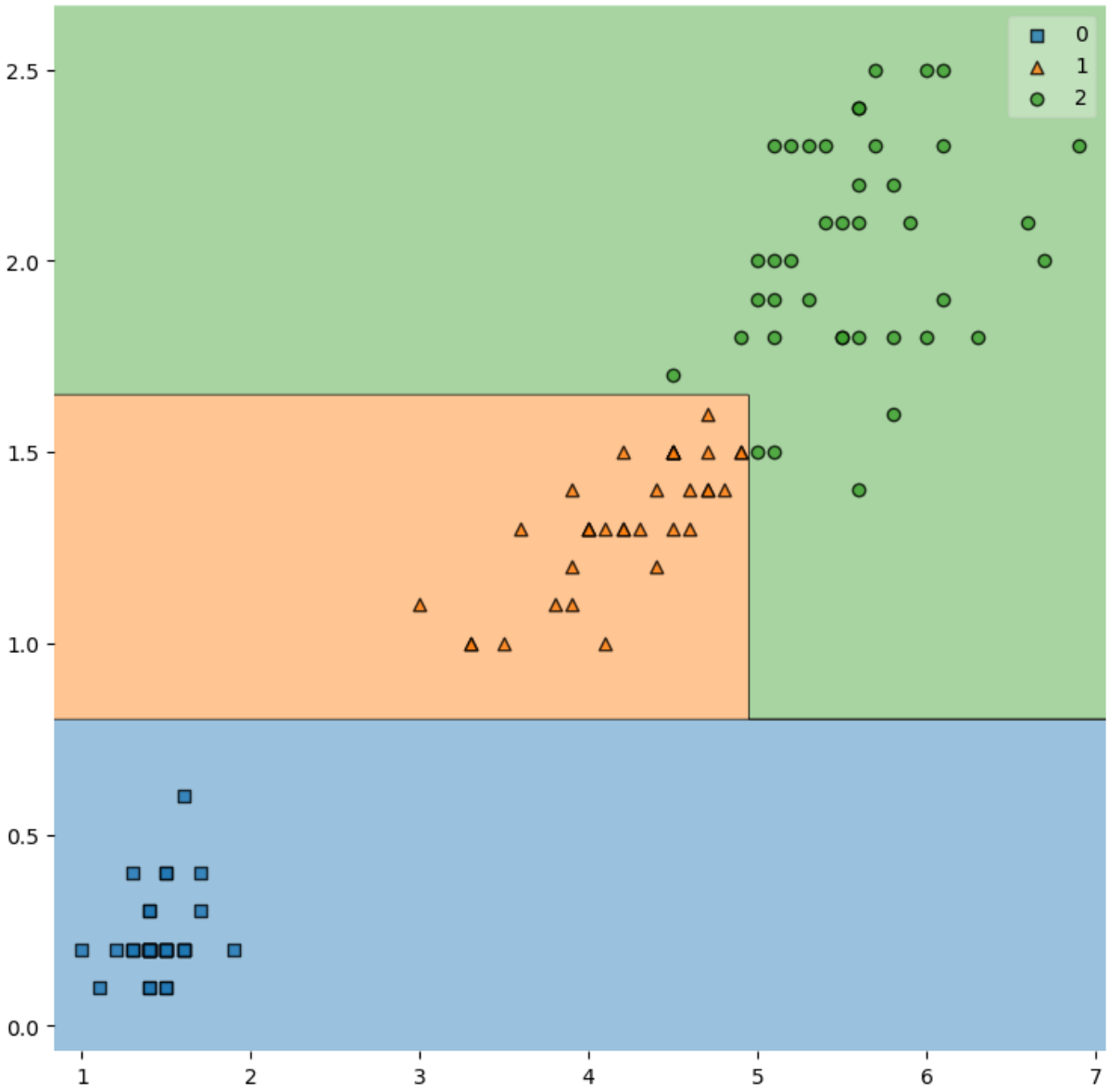
Or use some helper methods or libraries to plot the *decision regions*:

N.B.: The library we use in this demonstration only works with numerical values, hence the cumbersome transformations.

In [60]:
```
from mlxtend.plotting import plot_decision_regions

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=21)

dt_model = DecisionTreeClassifier()
dt_model.fit(X_train.iloc[:,2:], np.unique(y_train, return_inverse=True)[1])

plt.figure(figsize=(9,9))
plot_decision_regions(X=X_train.iloc[:,2:].to_numpy(dtype=float), y=np.uniqu
```

```
/home/markus/miniconda3/lib/python3.13/site-packages/sklearn/utils/validatio
n.py:2749: UserWarning: X does not have valid feature names, but DecisionTree
Classifier was fitted with feature names
  warnings.warn(
```

Out[60]: <Axes: >



Decision *Th(r)eory*

Take a look at the provided article by Alan Jeffares!

Real Life Example: *Breast Cancer* Data Set

Time to apply our new knowledge to a real and meaningful example. According to the *Mayo Clinic*, breast cancer is the most common form of cancer diagnosed in women in the united states - but there's also some optimistic news:

> Substantial support for breast cancer awareness and research funding has helped created advances in the diagnosis and treatment of breast cancer. Breast cancer survival rates have increased, and the number of deaths associated with this disease is steadily declining, largely due to factors such as earlier detection, a new personalized approach to treatment and a better understanding of the disease.

Maybe the chances for early detection could be increased even further by *machine learning*?

For this demo we'll be working with the *Breast Cancer Wisconsin* dataset, containing features computed from a digitized image of a *fine needle aspirate* (*FNA*) of breast mass. They describe characteristics of the cell nuclei present in the image. Our *target* is the prediction whether the tissue sample is *malignant* or not.

To focus on the task at hand - the prediction itself - we'll limit ourselves to a quick peek at the data and skip the visualization for now:

In [61]:
```
bcw_data = pd.read_csv('breast-cancer-prepared.csv')
```

In [62]:
```
bcw_data.sample(10, random_state=my_random_state)
```

Out[62]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points
172	15.460	11.89	102.50	736.9	0.12570	0.15550	0.203200	0.1097(
553	9.333	21.94	59.01	264.0	0.09240	0.05605	0.039960	0.01282
374	13.690	16.07	87.84	579.1	0.08302	0.06374	0.025560	0.0203
370	16.350	23.29	109.00	840.4	0.09742	0.14970	0.181100	0.08773
419	11.160	21.41	70.95	380.3	0.10180	0.05978	0.008955	0.01076
152	9.731	15.34	63.78	300.2	0.10720	0.15990	0.410800	0.07857
550	10.860	21.48	68.51	360.5	0.07431	0.04227	0.000000	0.00000
92	13.270	14.76	84.74	551.7	0.07355	0.05055	0.032610	0.02648
264	NaN	22.07	111.60	928.3	0.09726	0.08995	0.090610	0.06527
214	14.190	23.81	92.87	610.7	0.09463	0.13060	0.111500	0.06462

10 rows × 31 columns

In [63]:
```
bcw_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   mean radius              567 non-null    float64
 1   mean texture             564 non-null    float64
 2   mean perimeter           566 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         560 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      558 non-null    float64
 8   mean symmetry            560 non-null    float64
 9   mean fractal dimension   562 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               544 non-null    float64
 14  smoothness error         569 non-null    float64
 15  compactness error        547 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           564 non-null    float64
 19  fractal dimension error  525 non-null    float64
 20  worst radius             516 non-null    float64
 21  worst texture            480 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               462 non-null    float64
 24  worst smoothness         569 non-null    float64
 25  worst compactness        569 non-null    float64
 26  worst concavity          569 non-null    float64
 27  worst concave points     502 non-null    float64
 28  worst symmetry           467 non-null    float64
 29  worst fractal dimension  569 non-null    float64
 30  is_malignant             569 non-null    int64  
dtypes: float64(30), int64(1)
memory usage: 137.9 KB
```

In [64]:
```
bcw_data.describe()
```

Out[64]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness
count	567.000000	564.000000	566.000000	569.000000	569.000000	560.000000
mean	14.122344	19.250656	91.905936	654.889104	0.096360	0.104689
std	3.527902	4.259503	24.345462	351.914129	0.014064	0.053012
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380
25%	11.695000	16.170000	75.065000	420.300000	0.086370	0.065403
50%	13.340000	18.835000	86.140000	551.100000	0.095870	0.094035
75%	15.780000	21.735000	104.025000	782.700000	0.105300	0.130500
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400

8 rows × 31 columns

Splitting And *Stratification*

As you've already learned, we can adjust *hyperparameters* to tune our model's performance. But how can we both tune and find out, which combination of hyperparameters (and preprocessing methods) performs best on unseen data?

- If we tweak and evaluate using the *training* data we risk *overfitting* - after all, we have no idea, if our model simply *memorizes the right answers*.
- If we tweak and evaluate using the *test* data we risk adapting to data, that should have remained unseen - preventing an unbiased evaluation of how well our model performs on unseen data.

This is where the additional *validation* set comes into play:

- We use the *training* data to train our model.
- We play around with various hyperparameters (and preprocessing methods) and evaluate the performance using the *validation* set.
- As soon as we have found the model that performs best on the validation set, we can use the unseen data in the *test* set for the final evaluation.

The following image (from *V7 Labs*) illustrates the general process:

As always, there is no *one size fits all* split percentage we can use, since it's hugely depending on the data (and amount of it). The following typical split percentages are suggested by *V7 Labs*:

We'll be using the 70%-15%-15%-split for this demo. For this we can reuse the `train_test_split` method by simply calling it two times:

In [65]:
```
my_test_and_val_ratio = 0.3
```

In [66]:
```
X = bcw_data[bcw_data.columns[:-1]]
y = bcw_data['is_malignant']
#0.3 => 70% training and 30% testing
X_train, X_test_full, y_train, y_test_full = train_test_split(X, y, random_s
#0.5 on the testing => 15% testing and 15% validation
X_val, X_test, y_val, y_test = train_test_split(X_test_full, y_test_full, ra
```

In [67]:
```
#just to be sure :) - let us check via the lenght of the datasets
pd.DataFrame([len(y_train), len(y_val), len(y_test)], index=['train', 'valid
```

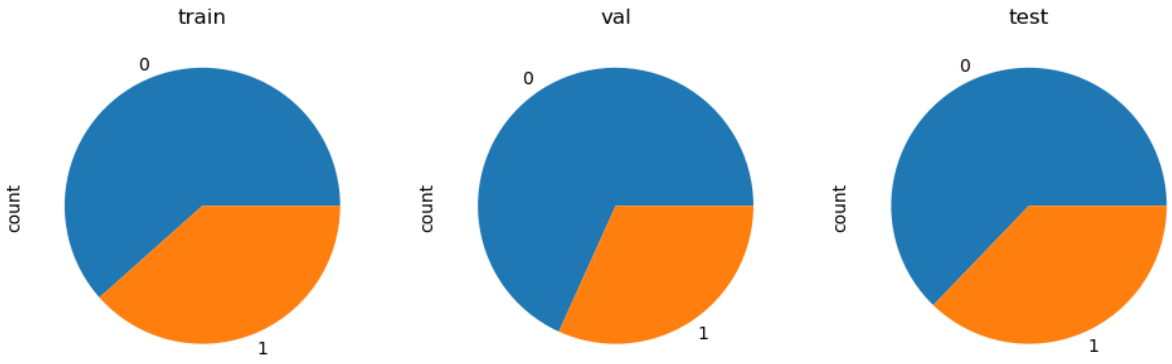
Out[67]:
```
array([<Axes: ylabel='0'>], dtype=object)
```

The above pie chart shows that we correctly split the data. Or did we? Is there anything else we need to look out for?

Let's take a look at the *class distributions* across the three sets - i.e. how many of the recorded tissue samples are malignant per set:

In [68]:
```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12,6))
y_train.value_counts().plot(kind='pie', ax=ax1, title='train')
y_val.value_counts().plot(kind='pie', ax=ax2, title='val')
y_test.value_counts().plot(kind='pie', ax=ax3, title='test')
```

Out[68]:
```
<Axes: title={'center': 'test'}, ylabel='count'>
```



As we can see, the classes are not distributed equally - in our case the validation set contains less malignant samples than the others. Depending on the random seed we use, this can drastically change - and drastically influence our machine learning model and its evaluation. E.g. imagine an extreme case, where the malignant samples are severly underrepresented in the training set: How should our model learn, how to

detect cancerous cells?

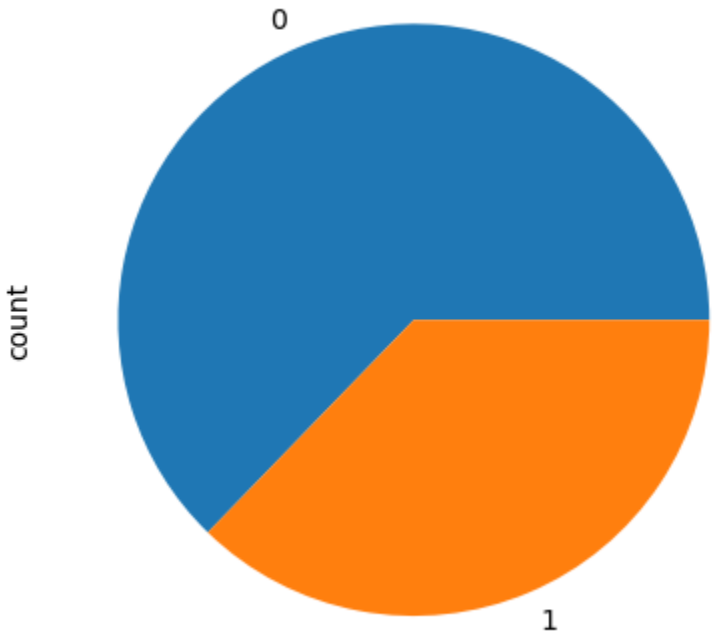
Let's take a look at the overall class distribution across all data:

In [69]:
```
bcw_data['is_malignant'].value_counts()
```

Out[69]:
```
is_malignant
0    357
1    212
Name: count, dtype: int64
```

In [70]:
```
bcw_data['is_malignant'].value_counts().plot(kind='pie')
```

Out[70]:
```
<Axes: ylabel='count'>
```



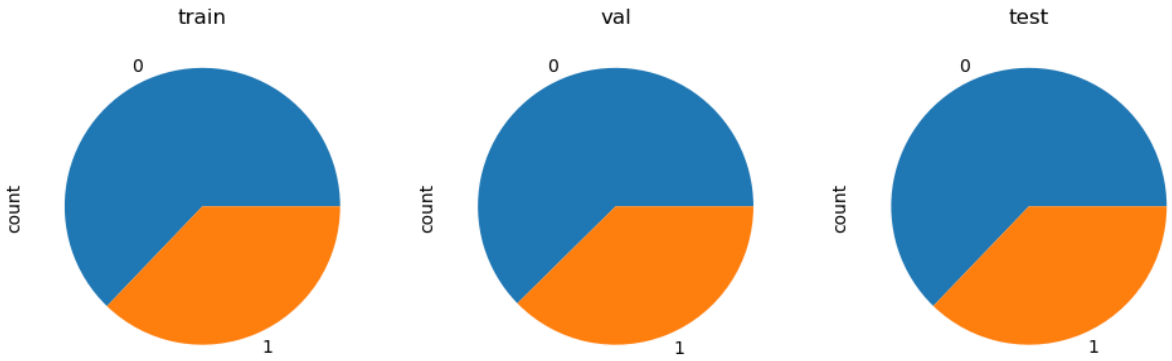
As you see we're dealing with a slightly *imbalanced* distribution of target classes. It makes sense to ensure the same proportions when splitting into *training* and *testing* data. This is called ***stratification*** (or *stratified sampling*) and can be easily achieved using the `stratify` parameter:

In [71]:
```
X = bcw_data[bcw_data.columns[:-1]]
y = bcw_data['is_malignant']

#same as before - this time with stratify!
X_train, X_test_full, y_train, y_test_full = train_test_split(X, y, random_s
X_val, X_test, y_val, y_test = train_test_split(X_test_full, y_test_full, ra
```

In [72]:
```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(12,6))
y_train.value_counts().plot(kind='pie', ax=ax1, title='train')
y_val.value_counts().plot(kind='pie', ax=ax2, title='val')
```

```
y_test.value_counts().plot(kind='pie', ax=ax3, title='test')
```

Out[72]: <Axes: title={'center': 'test'}, ylabel='count'>



Et voilà - the proportion of malignant samples is now the same across all sets. We can continue with the training.

Baseline

When starting a machine learning project, it's a good idea to define a *baseline* to compare against. We can easily achieve this using a `DummyClassifier`: In combination with the `most_frequent` parameter, it ignores all the features and simply returns the most often observed label. Since approximately two out of three tissue samples are benign, this is analogous to saying *every sample is benign* - resulting in an accuracy slightly above 60%:

In [73]:
```
from sklearn.dummy import DummyClassifier
```

In [74]:
```
dummy_model = DummyClassifier(strategy='most_frequent')
dummy_model.fit(X_train, y_train)

y_pred = dummy_model.predict(X_test)

accuracy_score = metrics.accuracy_score(y_test, y_pred)
print('Accuracy: {:.2%}'.format(accuracy_score))
```

```
Accuracy: 62.79%
```

First Try With A Decision Tree

Let's see if we can beat the baseline using a decision tree and all columns (but the label obviously) as features:

In [75]:
```
try:
    dt_model = DecisionTreeClassifier(random_state=my_random_state)
    dt_model.fit(X_train, y_train)

    y_pred = dt_model.predict(X_val)
```

```
    accuracy_score = metrics.accuracy_score(y_val, y_pred)
    print('Accuracy: {:.2%}'.format(accuracy_score))
except Exception as e:
    print(e)
```

```
Accuracy: 88.24%
```

Apparently the decision tree can't work with *null* values... let's see if and how we can fix that.

Dealing With Missing Values

In real-world data you'll often encounter missing values - be it due to problems when recording observations or data corruption. You need to learn how to handle these cases, since many machine learning algorithms can't work with missing data.

Dropping

An obvious possible solution is simply removing missing values. Depending on the amount, you can either drop the erroneous rows or even the whole columns. There's no exact ratio of missing values, when you should start considering dropping a whole column - literature suggests about 50%, but it highly depends on the dataset.

Let's take another look at the missing value count:

In [76]:
```
bcw_data.isnull().sum()
```

Out[76]:
```
mean radius                2
mean texture               5
mean perimeter             3
mean area                  0
mean smoothness            0
mean compactness           9
mean concavity             0
mean concave points       11
mean symmetry              9
mean fractal dimension     7
radius error               0
texture error              0
perimeter error            0
area error                25
smoothness error           0
compactness error         22
concavity error            0
concave points error       0
symmetry error             5
fractal dimension error   44
worst radius              53
worst texture             89
worst perimeter            0
worst area               107
worst smoothness           0
worst compactness          0
worst concavity            0
worst concave points      67
worst symmetry           102
worst fractal dimension    0
is_malignant               0
dtype: int64
```

Let's see how many rows are affected, when we simply drop them using the built-in dropna -method:

In [77]:
```
bcw_data_dropped_rows = bcw_data.dropna()
bcw_data_dropped_rows.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 9 entries, 27 to 545
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype  
---  ------                   --------------  -----  
 0   mean radius              9 non-null      float64
 1   mean texture             9 non-null      float64
 2   mean perimeter           9 non-null      float64
 3   mean area                9 non-null      float64
 4   mean smoothness          9 non-null      float64
 5   mean compactness         9 non-null      float64
 6   mean concavity           9 non-null      float64
 7   mean concave points      9 non-null      float64
 8   mean symmetry            9 non-null      float64
 9   mean fractal dimension   9 non-null      float64
 10  radius error             9 non-null      float64
 11  texture error            9 non-null      float64
 12  perimeter error          9 non-null      float64
 13  area error               9 non-null      float64
 14  smoothness error         9 non-null      float64
 15  compactness error        9 non-null      float64
 16  concavity error          9 non-null      float64
 17  concave points error     9 non-null      float64
 18  symmetry error           9 non-null      float64
 19  fractal dimension error  9 non-null      float64
 20  worst radius             9 non-null      float64
 21  worst texture            9 non-null      float64
 22  worst perimeter          9 non-null      float64
 23  worst area               9 non-null      float64
 24  worst smoothness         9 non-null      float64
 25  worst compactness        9 non-null      float64
 26  worst concavity          9 non-null      float64
 27  worst concave points     9 non-null      float64
 28  worst symmetry           9 non-null      float64
 29  worst fractal dimension  9 non-null      float64
 30  is_malignant             9 non-null      int64  
dtypes: float64(30), int64(1)
memory usage: 2.2 KB
```

Unfortunately this removed everything but 9 rows, rendering the dataset useless.

To recap: In a situation like the following example...

In [78]:
```
example_data = pd.DataFrame({'my_value': [1, 2, np.nan, 4], 'my_other_value'
example_data
```

Out[78]:

	my_value	my_other_value	yet_another_value
0	1.0	5	9.0
1	2.0	6	10.0
2	NaN	13	NaN
3	4.0	8	12.0

...deleting the row(s) containing missing values would definitely make sense.

Our situation is closer to the next example...

In [79]:
```
example_data = pd.DataFrame({'my_value': [1, 2, 3, 4], 'my_other_value': [5,
example_data
```

Out[79]:

	my_value	my_other_value	yet_another_value
0	1	5.0	9.0
1	2	6.0	NaN
2	3	NaN	11.0
3	4	8.0	NaN

...where we would lose almost all rows.

Let's try dropping the columns instead. We start by gathering the problematic columns:

In [80]:
```
cols_with_missing_vals = [c for c in X.columns if X[c].isnull().any()]
cols_with_missing_vals
```

Out[80]:
```
['mean radius',
 'mean texture',
 'mean perimeter',
 'mean compactness',
 'mean concave points',
 'mean symmetry',
 'mean fractal dimension',
 'area error',
 'compactness error',
 'symmetry error',
 'fractal dimension error',
 'worst radius',
 'worst texture',
 'worst area',
 'worst concave points',
 'worst symmetry']
```

We can now use the `drop` -method in combination with the `axis` parameter set to 1 (since we want to delete *columns*). We obviously need to do this on training, validation and testing data:

In [81]:
```
X_train_dropped_cols = X_train.drop(cols_with_missing_vals, axis=1)
X_val_dropped_cols = X_val.drop(cols_with_missing_vals, axis=1)
X_test_dropped_cols = X_test.drop(cols_with_missing_vals, axis=1)
```

We can finally train our tree:

In [82]:
```
dt_model = DecisionTreeClassifier(random_state=my_random_state)
```

```
dt_model.fit(X_train_dropped_cols, y_train)

y_pred = dt_model.predict(X_val_dropped_cols)

accuracy_score = metrics.accuracy_score(y_val, y_pred)
print('Accuracy: {:.2%}'.format(accuracy_score))
```

```
Accuracy: 83.53%
```

Hyperparameter Optimization

Not bad, we're far above the baseline! Let's see if we can tune the *hyperparameters* to achieve an even better result. We're gonna focus on the *tree depth* in the following *helper method*:

In [83]:
```
def train_and_find_best_depth(X_train, X_val, y_train, y_val, do_print):
    result = None
    accuracy_max = -1

    for curr_max_depth in range(1, 20):
        dt_model = DecisionTreeClassifier(max_depth=curr_max_depth, random_s
        dt_model.fit(X_train, y_train)

        y_pred = dt_model.predict(X_val)

        accuracy_score = metrics.accuracy_score(y_val, y_pred)

        if accuracy_score >= accuracy_max:
            accuracy_max = accuracy_score
            result = curr_max_depth

        if do_print:
            print('max depth {}: {:.2%} accuracy on validation set.'.format(

    if do_print:
        print('-' * 20)
        print('best max depth {} has {:.2%} accuracy.'.format(result, accura

    return result
```

And this is the moment, where the *validation* set comes into play. We can now use it to find the best tree depth:

In [84]:
```
best_max_depth = train_and_find_best_depth(X_train_dropped_cols, X_val_dropp
```

```
max depth 1: 89.41% accuracy on validation set.
max depth 2: 92.94% accuracy on validation set.
max depth 3: 88.24% accuracy on validation set.
max depth 4: 89.41% accuracy on validation set.
max depth 5: 89.41% accuracy on validation set.
max depth 6: 89.41% accuracy on validation set.
max depth 7: 85.88% accuracy on validation set.
max depth 8: 88.24% accuracy on validation set.
max depth 9: 83.53% accuracy on validation set.
max depth 10: 83.53% accuracy on validation set.
max depth 11: 83.53% accuracy on validation set.
max depth 12: 83.53% accuracy on validation set.
max depth 13: 83.53% accuracy on validation set.
max depth 14: 83.53% accuracy on validation set.
max depth 15: 83.53% accuracy on validation set.
max depth 16: 83.53% accuracy on validation set.
max depth 17: 83.53% accuracy on validation set.
max depth 18: 83.53% accuracy on validation set.
max depth 19: 83.53% accuracy on validation set.
--------------------
best max depth 2 has 92.94% accuracy.
```

Apparently the *max depth* of 2 performs best on the validation set. Let's see, how well it performs on the unseen data in the *test* set:

In [85]:
```
dt_model = DecisionTreeClassifier(max_depth=best_max_depth, random_state=my_
dt_model.fit(X_train_dropped_cols, y_train)

y_pred = dt_model.predict(X_test_dropped_cols)

accuracy_score_dropped_cols = metrics.accuracy_score(y_test, y_pred)
print('Accuracy: {:.2%}'.format(accuracy_score_dropped_cols))
```

```
Accuracy: 89.53%
```

Imputing Numerical Data

A downside of above approach is the loss of potentially important data, especially in columns where only few values were missing. A simple and popular alternative is ***imputation*** - the process of replacing missing data with substituted values.

Using statistical methods - e.g. the *mean*, *median*, *minimum*, *maximum* or *most frequent* value of a column - for estimating a suitable replacement and replacing all missing values with it is fast to calculate and often proves (very) effective.

Let's take a look at an example using a small dataframe with some missing values:

In [86]:
```
example_data = pd.DataFrame({'my_value': [1, 2, np.nan, 6], 'my_other_value'
example_data
```

Out[86]:

	my_value	my_other_value
0	1.0	NaN
1	2.0	4.0
2	NaN	NaN
3	6.0	10.0

We now use *scikit-learn*'s `SimpleImputer` for imputing the missing values: After importing the class and setting the *strategy* (i.e. the (statistical) method), we ***fit*** the *imputer* to our data so it can find suitable replacement values for each column. Afterwards we use it to ***transform*** the columns, i.e. replace all missing values.

In [87]:
```
from sklearn.impute import SimpleImputer
```

In [88]:
```
simple_imputer = SimpleImputer(strategy='mean')

simple_imputer.fit(example_data[['my_value', 'my_other_value']])
example_data[['my_value', 'my_other_value']] = simple_imputer.transform(exam

example_data
```

Out[88]:

	my_value	my_other_value
0	1.0	7.0
1	2.0	4.0
2	3.0	7.0
3	6.0	10.0

Let's try it out on our dataset. We already identified problematic columns above:

In [89]:
```
cols_with_missing_vals
```

Out[89]:
```
['mean radius',
 'mean texture',
 'mean perimeter',
 'mean compactness',
 'mean concave points',
 'mean symmetry',
 'mean fractal dimension',
 'area error',
 'compactness error',
 'symmetry error',
 'fractal dimension error',
 'worst radius',
 'worst texture',
 'worst area',
 'worst concave points',
 'worst symmetry']
```

We now use the `SimpleImputer` for dealing with all those columns:

In [90]:
```
simple_imputer = SimpleImputer(strategy='mean')

#deep copy - remember why?
X_train_imputed = X_train.copy()
X_val_imputed = X_val.copy()
X_test_imputed = X_test.copy()

X_train_imputed[cols_with_missing_vals] = simple_imputer.fit_transform(X_tra
X_val_imputed[cols_with_missing_vals] = simple_imputer.transform(X_val[cols_
X_test_imputed[cols_with_missing_vals] = simple_imputer.transform(X_test[col
```

As you saw, we combined the *two-step-approach* from above to perform both *fitting* and *transforming* on the training data using `fit_transform` . Why don't we use the same method on the test data? In order to simulate real world scenarios, we want the test data to be a completely new surprise set for our model! This is why we only use the imputation parameters learned from (or *fitted to*) the training data.

For this reason the standard approach is:

1. `fit_transform` on **training** data.
2. `transform` on **test/validation** data.

Let's see if the imputation improved our accuracy:

In [91]:
```
best_max_depth = train_and_find_best_depth(X_train_imputed, X_val_imputed, y
```

```
max depth 1: 89.41% accuracy on validation set.
max depth 2: 91.76% accuracy on validation set.
max depth 3: 89.41% accuracy on validation set.
max depth 4: 89.41% accuracy on validation set.
max depth 5: 91.76% accuracy on validation set.
max depth 6: 90.59% accuracy on validation set.
max depth 7: 83.53% accuracy on validation set.
max depth 8: 84.71% accuracy on validation set.
max depth 9: 84.71% accuracy on validation set.
max depth 10: 84.71% accuracy on validation set.
max depth 11: 84.71% accuracy on validation set.
max depth 12: 84.71% accuracy on validation set.
max depth 13: 84.71% accuracy on validation set.
max depth 14: 84.71% accuracy on validation set.
max depth 15: 84.71% accuracy on validation set.
max depth 16: 84.71% accuracy on validation set.
max depth 17: 84.71% accuracy on validation set.
max depth 18: 84.71% accuracy on validation set.
max depth 19: 84.71% accuracy on validation set.
--------------------
best max depth 5 has 91.76% accuracy.
```

In [92]:
```
dt_model = DecisionTreeClassifier(max_depth=best_max_depth, random_state=my_
dt_model.fit(X_train_imputed, y_train)
```

```
y_pred = dt_model.predict(X_test_imputed)

accuracy_score_imputed = metrics.accuracy_score(y_test, y_pred)
print('Accuracy: {:.2%}'.format(accuracy_score_imputed))
```

```
Accuracy: 91.86%
```

Albeit small, we still see a slight improvement. Depending on the dataset, *imputation* will drastically boost the results in some cases, while in others it doesn't help that much.

Imputing With A Flag

Imputed values may be systematically above or below their actual values (which weren't collected in the dataset). Or rows with missing values may be unique in some other way. In that case, your model would make better predictions by considering which values were originally missing.

An *extension* to simple imputation would be adding a *flag* column, indicating where values were missing.

Let's try this approach:

In [93]:
```
simple_imputer = SimpleImputer(strategy='mean')

X_train_imputed_plus = X_train.copy()
X_val_imputed_plus = X_val.copy()
X_test_imputed_plus = X_test.copy()

for curr_col in cols_with_missing_vals:
    X_train_imputed_plus[curr_col + '_was_missing'] = X_train_imputed_plus[c
    X_val_imputed_plus[curr_col + '_was_missing'] = X_val_imputed_plus[curr_
    X_test_imputed_plus[curr_col + '_was_missing'] = X_test_imputed_plus[cur

X_train_imputed_plus[cols_with_missing_vals] = simple_imputer.fit_transform(
X_val_imputed_plus[cols_with_missing_vals] = simple_imputer.transform(X_val_
X_test_imputed_plus[cols_with_missing_vals] = simple_imputer.transform(X_tes
```

In [94]:
```
X_train_imputed_plus
```

Out[94]:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mea concav point
515	11.34	18.61	72.76	391.2	0.10490	0.08499	0.04302	0.0259
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.0000
47	13.17	18.66	85.98	534.6	0.11580	0.12310	0.12260	0.0734
485	12.45	16.41	82.85	476.7	0.09514	0.15110	0.15440	0.0484
211	11.84	18.94	75.51	428.0	0.08871	0.06900	0.02669	0.0139
...	...	...	...	...	...	...	...	.
27	18.61	20.25	122.10	1094.0	0.09440	0.10660	0.14900	0.0773
214	14.19	23.81	92.87	610.7	0.09463	0.13060	0.11150	0.0646
506	12.22	20.04	79.47	453.1	0.10960	0.11520	0.08175	0.0216
155	12.25	17.94	78.27	460.3	0.08654	0.06679	0.03885	0.0233
262	17.29	22.13	114.40	947.8	0.08999	0.12730	0.09697	0.0750

398 rows × 46 columns

In [95]:
```
best_max_depth = train_and_find_best_depth(X_train_imputed_plus, X_val_imput
```

```
max depth 1: 89.41% accuracy on validation set.
max depth 2: 91.76% accuracy on validation set.
max depth 3: 88.24% accuracy on validation set.
max depth 4: 90.59% accuracy on validation set.
max depth 5: 91.76% accuracy on validation set.
max depth 6: 89.41% accuracy on validation set.
max depth 7: 89.41% accuracy on validation set.
max depth 8: 89.41% accuracy on validation set.
max depth 9: 89.41% accuracy on validation set.
max depth 10: 89.41% accuracy on validation set.
max depth 11: 89.41% accuracy on validation set.
max depth 12: 89.41% accuracy on validation set.
max depth 13: 89.41% accuracy on validation set.
max depth 14: 89.41% accuracy on validation set.
max depth 15: 89.41% accuracy on validation set.
max depth 16: 89.41% accuracy on validation set.
max depth 17: 89.41% accuracy on validation set.
max depth 18: 89.41% accuracy on validation set.
max depth 19: 89.41% accuracy on validation set.
--------------------
best max depth 5 has 91.76% accuracy.
```

In [96]:
```
dt_model = DecisionTreeClassifier(max_depth=best_max_depth, random_state=my_
dt_model.fit(X_train_imputed_plus, y_train)

y_pred = dt_model.predict(X_test_imputed_plus)

accuracy_score_imputed_plus = metrics.accuracy_score(y_test, y_pred)
```

```
print('Accuracy: {:.2%}'.format(accuracy_score_imputed_plus))
```

```
Accuracy: 94.19%
```

That boosted the accuracy on our data even further!

Conclusion

In this demonstration you not only got to know the decision tree algorithm, you should be also starting to recognize the importance of data cleaning and preprocessing - which can take up to 80% of the overall effort of a data science project.

	Accuracy with best *max depth*
Baseline.	62.79%
Columns with missing values dropped.	89.53%
Imputation of missing values.	91.86%
Imputation of missing values with a flag.	94.19%

In our example we were able to increase the models performance with imputation, as you can see in the table above. Note that the methods you need to use and the results will greatly vary depending on your data!

Obviously, the `random_state` also plays a major role (try it out!). We'll see how to tackle this problem in one of the next lessons.

In [97]:
```
import time
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics

data_variants = {
    "Dropped Columns": (X_train_dropped_cols, X_val_dropped_cols, X_test_dro
    "Imputation": (X_train_imputed, X_val_imputed, X_test_imputed),
    "Imputation + Flag": (X_train_imputed_plus, X_val_imputed_plus, X_test_i
}

criterion_options = ['gini', 'entropy']

results_store = {}

for variant_name, (train_X, val_X, test_X) in data_variants.items():
    results_store[variant_name] = {}
    print(f"\n=== Datensatz: {variant_name} ===")

    for criterion_type in criterion_options:
        t_start = time.time()

        # Training
        model = DecisionTreeClassifier(
```

```
            criterion=criterion_type,
            random_state=my_random_state
        )
        model.fit(train_X, y_train)

        # Prediction auf Validation-Set
        val_predictions = model.predict(val_X)
        val_accuracy = metrics.accuracy_score(y_val, val_predictions)

        # Zeitmessung
        runtime = time.time() - t_start

        # Ergebnisse speichern
        results_store[variant_name][criterion_type] = {
            'accuracy_val': val_accuracy,
            'time_sec': runtime
        }

        print(f"{criterion_type}: Accuracy={val_accuracy:.2%}, Trainingszeit

print("\n=== Test-Set Accuracy für beste Kriterien pro Datensatz ===")

for variant_name, criterion_results in results_store.items():
    best_criterion = max(
        criterion_results,
        key=lambda c: criterion_results[c]['accuracy_val']
    )

    train_X, val_X, test_X = data_variants[variant_name]

    model = DecisionTreeClassifier(
        criterion=best_criterion,
        random_state=my_random_state
    )
    model.fit(train_X, y_train)

    test_predictions = model.predict(test_X)
    test_accuracy = metrics.accuracy_score(y_test, test_predictions)

    print(f"{variant_name} - Best Criterion: {best_criterion}, Test Accuracy
```

```
=== Datensatz: Dropped Columns ===
gini: Accuracy=83.53%, Trainingszeit=0.0060s
entropy: Accuracy=91.76%, Trainingszeit=0.0042s

=== Datensatz: Imputation ===
gini: Accuracy=84.71%, Trainingszeit=0.0071s
entropy: Accuracy=94.12%, Trainingszeit=0.0062s

=== Datensatz: Imputation + Flag ===
gini: Accuracy=89.41%, Trainingszeit=0.0076s
entropy: Accuracy=94.12%, Trainingszeit=0.0061s

=== Test-Set Accuracy für beste Kriterien pro Datensatz ===
Dropped Columns - Best Criterion: entropy, Test Accuracy=89.53%
Imputation - Best Criterion: entropy, Test Accuracy=93.02%
Imputation + Flag - Best Criterion: entropy, Test Accuracy=91.86%
```

RANDOM FOREST

In [98]:

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn import metrics
import time

# verschiedene vorbereitete Datensatzvarianten
data_sets = {
    "Dropped Columns": (X_train_dropped_cols, X_val_dropped_cols, X_test_dro
    "Imputation": (X_train_imputed, X_val_imputed, X_test_imputed),
    "Imputation + Flag": (X_train_imputed_plus, X_val_imputed_plus, X_test_i
}

rf_results = {}

tree_count = 100  # wie viele Entscheidungsbäume im Random Forest verwendet

# jede Datensatzvariante einzeln evaluieren
for dataset_label, (train_data, val_data, test_data) in data_sets.items():

    rf_results[dataset_label] = {}
    print(f"\n=== Datensatz: {dataset_label} ===")

    # mögliche Split-Kriterien
    for criterion_choice in ["gini", "entropy"]:

        timer_start = time.time()

        # Modell erzeugen
        forest = RandomForestClassifier(
            n_estimators=tree_count,
            criterion=criterion_choice,
            random_state=my_random_state,
            n_jobs=-1  # parallele Nutzung aller CPU-Kerne
        )

        # Modell trainieren
```

```
        forest.fit(train_data, y_train)

        # Validation-Set auswerten
        predictions_val = forest.predict(val_data)
        val_acc = metrics.accuracy_score(y_val, predictions_val)

        duration = time.time() - timer_start

        # Ergebnis speichern
        rf_results[dataset_label][criterion_choice] = {
            "accuracy_val": val_acc,
            "time_sec": duration
        }

        print(f"{criterion_choice}: Accuracy={val_acc:.2%}, Trainingszeit={d


# =========================
# Bestes Modell pro Datensatz auf Test-Set prüfen
# =========================

print("\n=== Test-Set Accuracy für beste Kriterien pro Datensatz ===")

for dataset_label, criterion_data in rf_results.items():

    # bestes Kriterium anhand der Validation-Accuracy bestimmen
    best_criterion = max(
        criterion_data,
        key=lambda k: criterion_data[k]["accuracy_val"]
    )

    train_data, val_data, test_data = data_sets[dataset_label]

    forest = RandomForestClassifier(
        n_estimators=tree_count,
        criterion=best_criterion,
        random_state=my_random_state,
        n_jobs=-1
    )

    # Training erneut durchführen
    forest.fit(train_data, y_train)

    # finale Bewertung mit Testdaten
    predictions_test = forest.predict(test_data)
    test_acc = metrics.accuracy_score(y_test, predictions_test)

    print(f"{dataset_label} - Best Criterion: {best_criterion}, Test Accurac
```

```
=== Datensatz: Dropped Columns ===
gini: Accuracy=95.29%, Trainingszeit=0.1220s
entropy: Accuracy=94.12%, Trainingszeit=0.1175s

=== Datensatz: Imputation ===
gini: Accuracy=97.65%, Trainingszeit=0.1223s
entropy: Accuracy=96.47%, Trainingszeit=0.1347s

=== Datensatz: Imputation + Flag ===
gini: Accuracy=96.47%, Trainingszeit=0.1301s
entropy: Accuracy=96.47%, Trainingszeit=0.1231s

=== Test-Set Accuracy für beste Kriterien pro Datensatz ===
Dropped Columns - Best Criterion: gini, Test Accuracy=94.19%
Imputation - Best Criterion: gini, Test Accuracy=91.86%
Imputation + Flag - Best Criterion: gini, Test Accuracy=93.02%
```